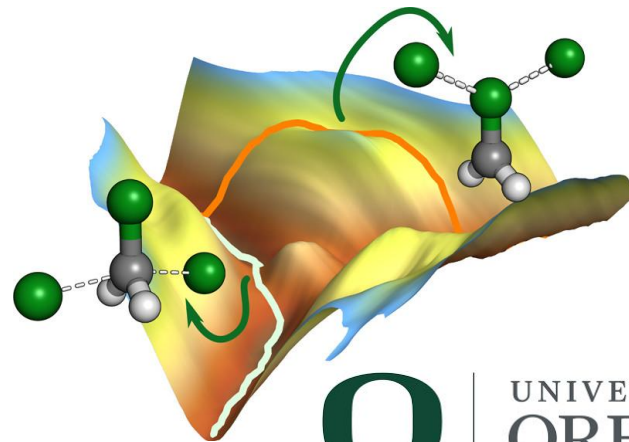
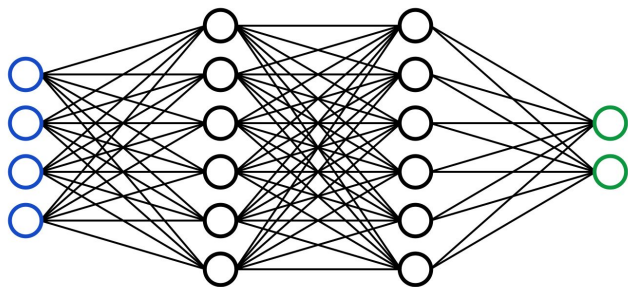


Machine Learning for Chemistry

Winter 2025

Topic 8: Neural Networks and Deep Learning



UNIVERSITY OF
OREGON

Instructor: Dr. Dhiman Ray

Assistant Professor, Department of Chemistry and Biochemistry

Overview of Topics

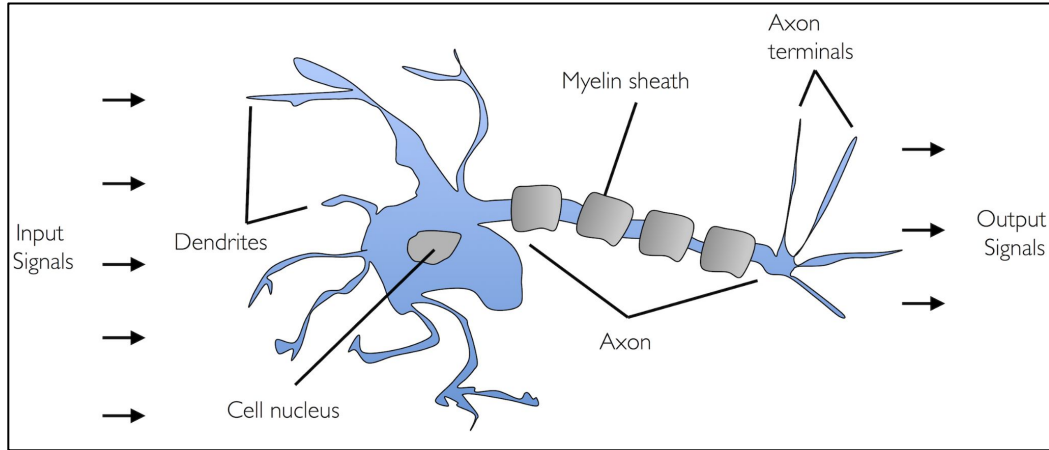
- Artificial Neuron and Perceptron
- Feedforward Neural Network
- Training of Neural Network with Backpropagation
- Dimensionality Reduction using Neural Network

References:

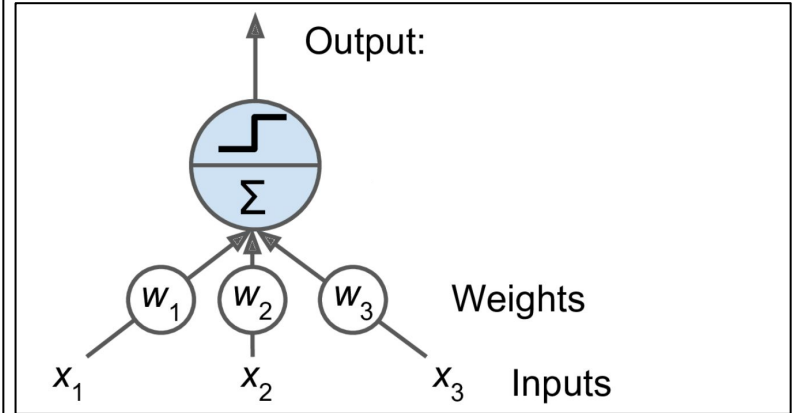
1. **Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow** by Aurélien Géron (Chapter 10)
2. **Machine Learning with Pytorch and Scikit-Learn** by Raschka, Liu, and Mirjalili (Chapter 2 and 11)

Biological vs Artificial Neuron

Biological Neuron



Artificial Neuron



Biological vs Artificial Neuron

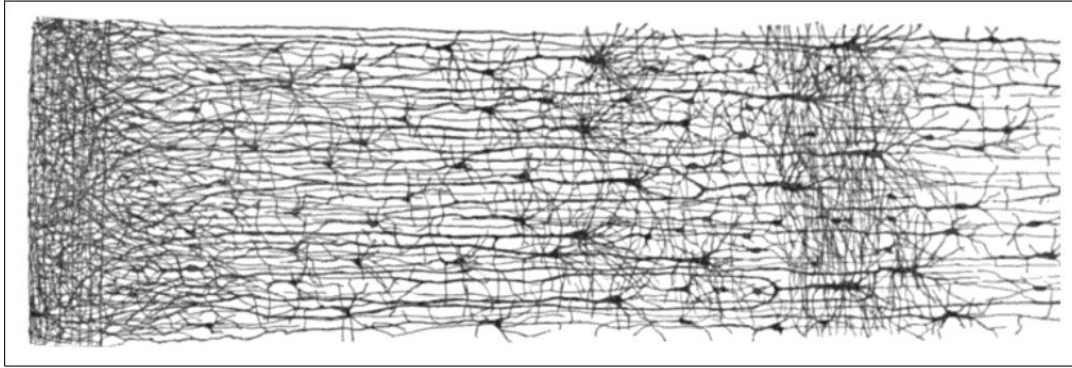
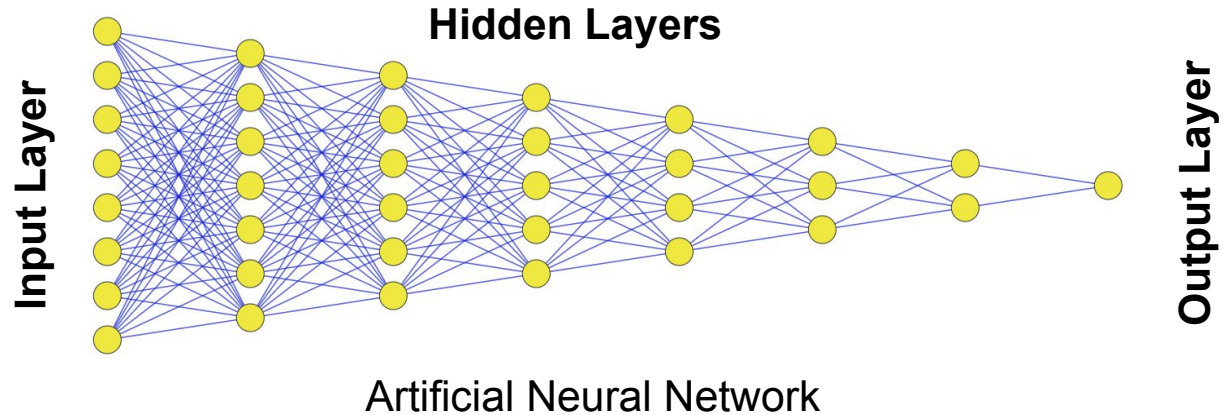
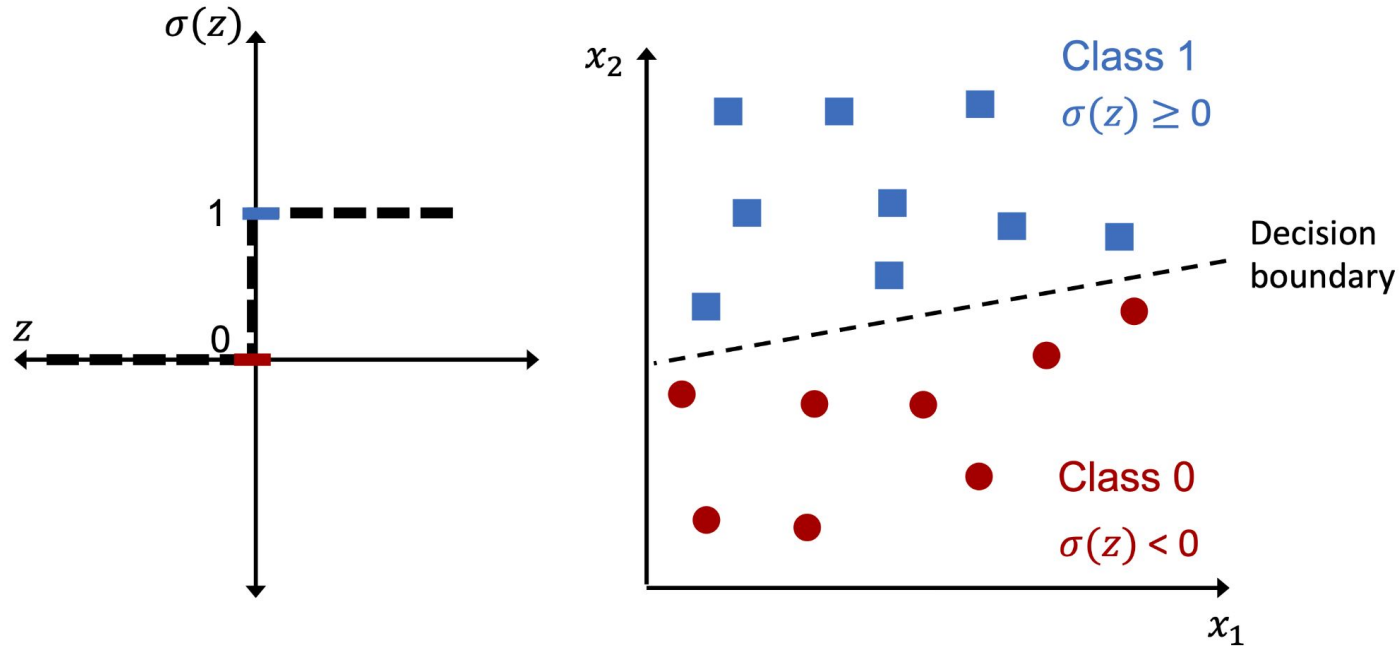


Figure 10-2. Multiple layers in a biological neural network (human cortex)⁵

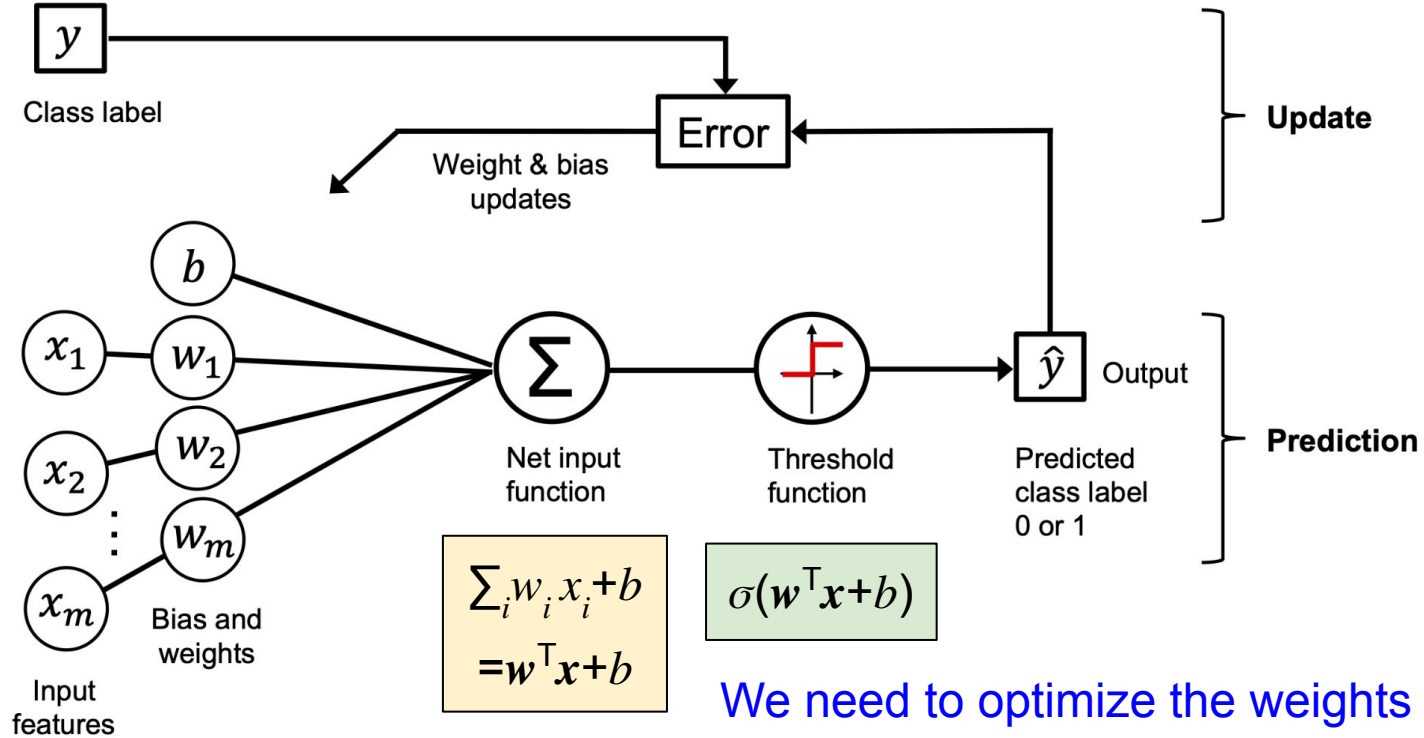


Single Layer Perceptron

Let's consider the binary classification problem



Single Layer Perceptron



We need to optimize the weights (w_i) and bias (b) to predict correct class labels

Perceptron Learning Rule

The weights and biases are updated in the following manner:

$$\begin{aligned}w_j &:= w_j + \Delta w_j & \Delta w_j &= \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)} \\b &:= b + \Delta b & \Delta b &= \eta(y^{(i)} - \hat{y}^{(i)})\end{aligned}$$

Notice the similarity with gradient descent in regression:

$$\theta_k^{(nextstep)} = \theta_k - \eta \cdot \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right) \cdot x_k^{(i)}$$

Perceptron Learning Rule

Right prediction: weights and biases are unchanged

$$\begin{aligned} (1) \quad & \boxed{y^{(i)} = 0, \quad \hat{y}^{(i)} = 0,} \quad \Delta w_j = \eta(0 - 0)x_j^{(i)} = 0, \quad \Delta b = \eta(0 - 0) = 0 \\ (2) \quad & \boxed{y^{(i)} = 1, \quad \hat{y}^{(i)} = 1,} \quad \Delta w_j = \eta(1 - 1)x_j^{(i)} = 0, \quad \Delta b = \eta(1 - 1) = 0 \end{aligned}$$

Perceptron Learning Rule

Right prediction: weights and biases are unchanged

$$(1) \quad \boxed{y^{(i)} = 0, \quad \hat{y}^{(i)} = 0,} \quad \Delta w_j = \eta(0 - 0)x_j^{(i)} = 0, \quad \Delta b = \eta(0 - 0) = 0$$

$$(2) \quad \boxed{y^{(i)} = 1, \quad \hat{y}^{(i)} = 1,} \quad \Delta w_j = \eta(1 - 1)x_j^{(i)} = 0, \quad \Delta b = \eta(1 - 1) = 0$$

Wrong prediction: weights and biases are moved toward the correct target value

$$(3) \quad \boxed{y^{(i)} = 1, \quad \hat{y}^{(i)} = 0,} \quad \Delta w_j = \eta(1 - 0)x_j^{(i)} = \eta x_j^{(i)}, \quad \Delta b = \eta(1 - 0) = \eta$$

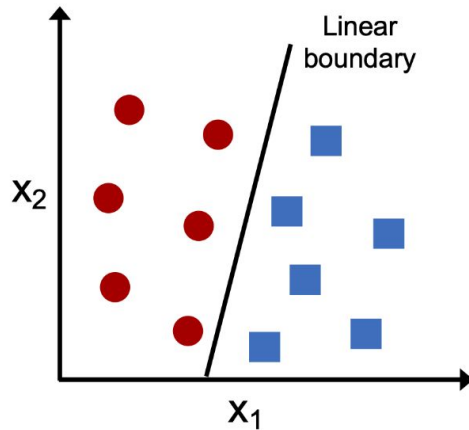
$$(4) \quad \boxed{y^{(i)} = 0, \quad \hat{y}^{(i)} = 1,} \quad \Delta w_j = \eta(0 - 1)x_j^{(i)} = -\eta x_j^{(i)}, \quad \Delta b = \eta(0 - 1) = -\eta$$

Limitation of Single Layer Perceptron

Single layer linear perceptron cannot discriminate between states with non-linear decision boundary

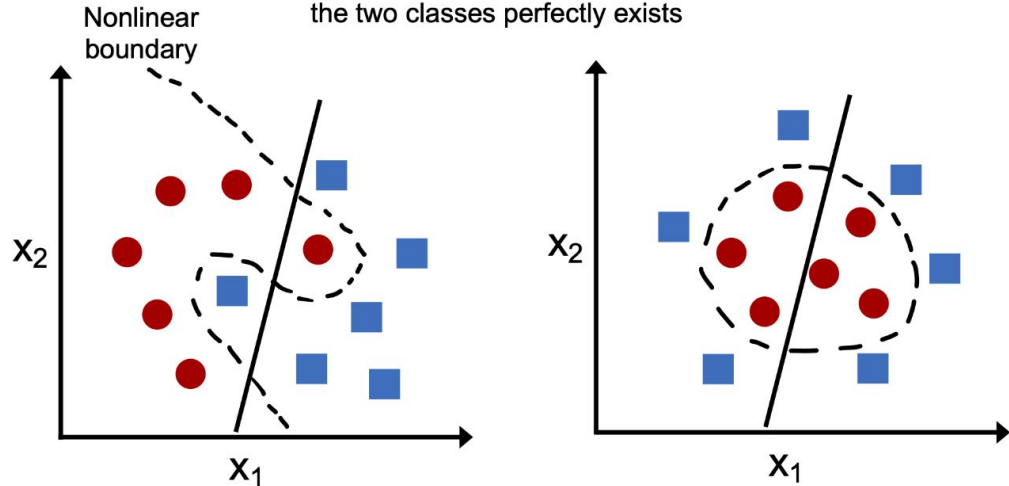
Linearly separable

A linear decision boundary that separates the two classes exists



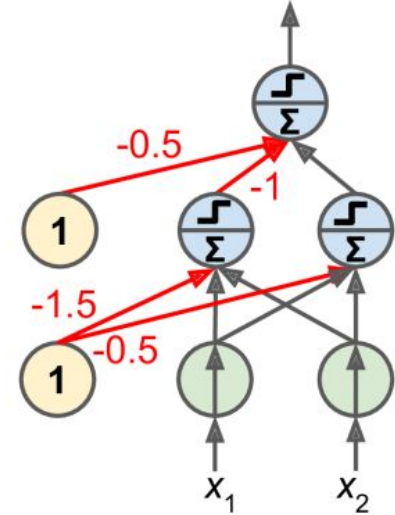
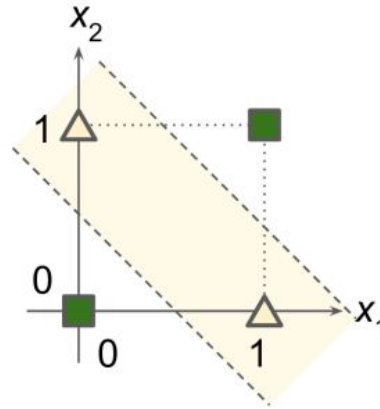
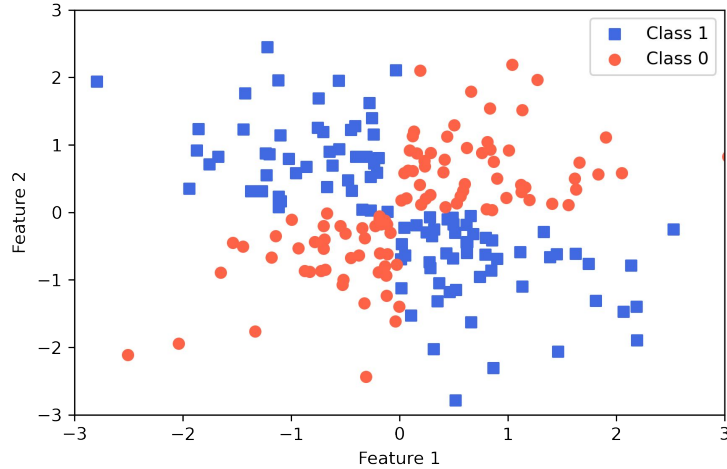
Not linearly separable

No linear decision boundary that separates the two classes perfectly exists



Multi-Layer Perceptron

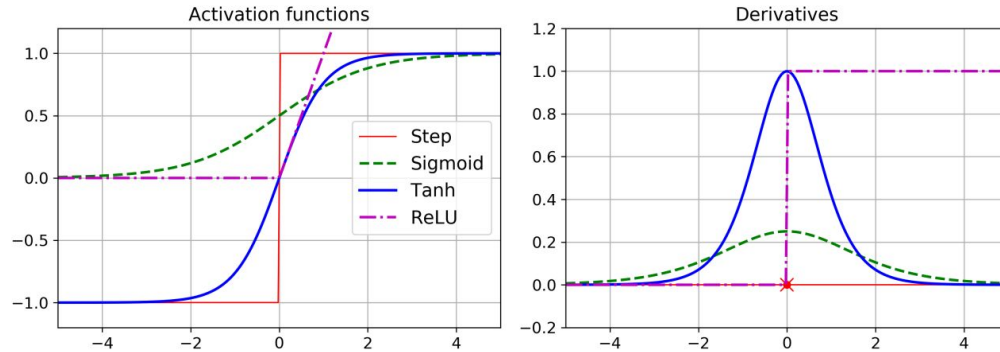
Multi-layer Perceptron can overcome this problem in certain situations for example the famous XOR classification problem



Multi-Layer Neural Network

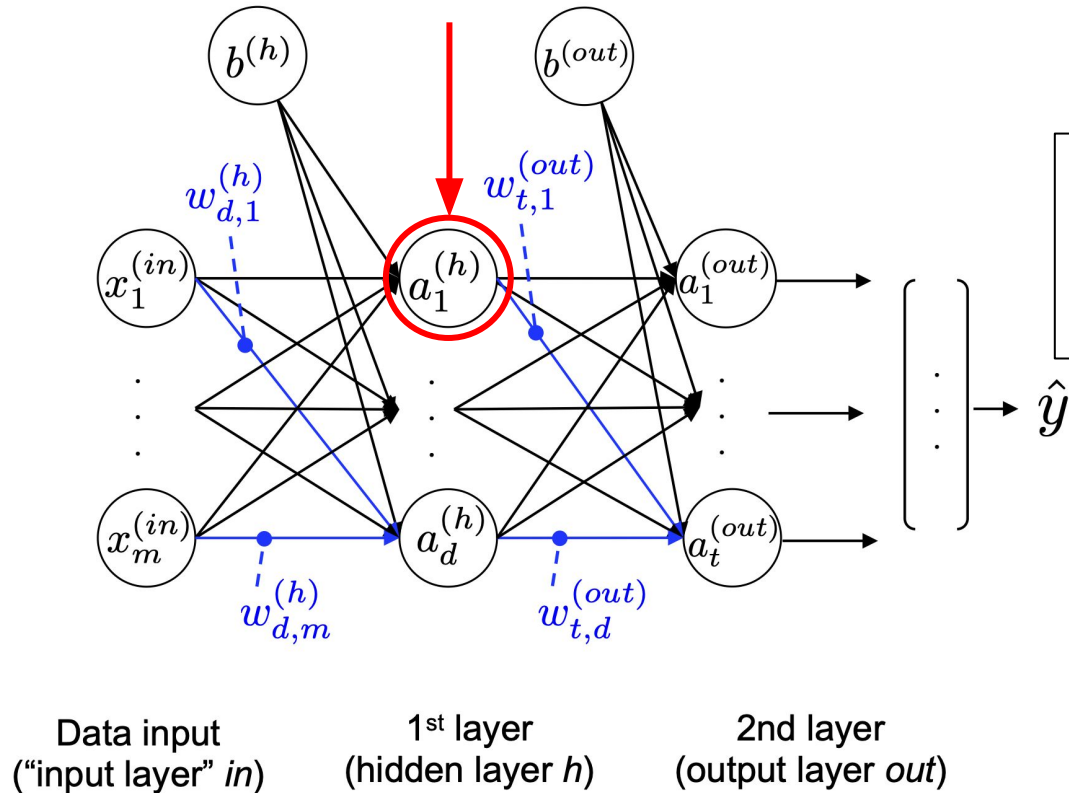
But how about more complex nonlinear functions in higher dimensions?

Replace the threshold function from step function to non-linear activations functions



Activation function	Equation	Example	1D graph
Linear	$\sigma(z) = z$	Adaline, linear regression	
Unit step (Heaviside function)	$\sigma(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\sigma(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise linear	$\sigma(z) = \begin{cases} 0 & z \leq -\frac{1}{2} \\ z + \frac{1}{2} & -\frac{1}{2} \leq z \leq \frac{1}{2} \\ 1 & z \geq \frac{1}{2} \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\sigma(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, multilayer NN	
Hyperbolic tangent (tanh)	$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\sigma(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

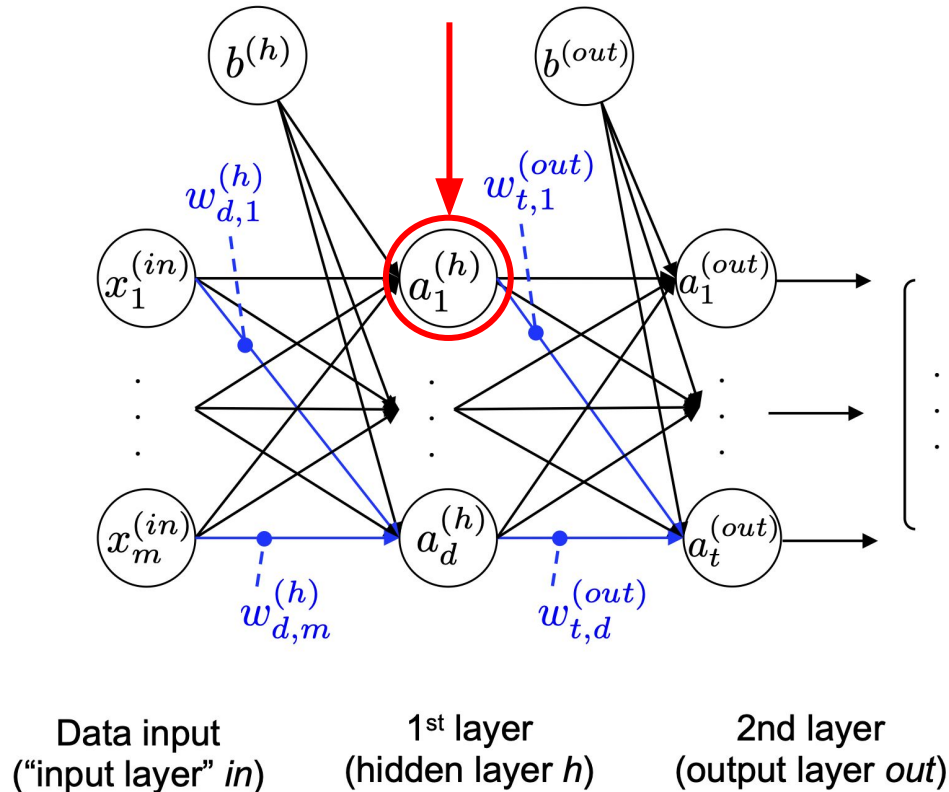
Multi-Layer Neural Network



- Create linear combination
- Apply activation function

$$z_1^{(h)} = x_1^{(in)} w_{1,1}^{(h)} + x_2^{(in)} w_{1,2}^{(h)} + \dots + x_m^{(in)} w_{1,m}^{(h)}$$
$$a_1^{(h)} = \sigma(z_1^{(h)})$$

Multi-Layer Neural Network



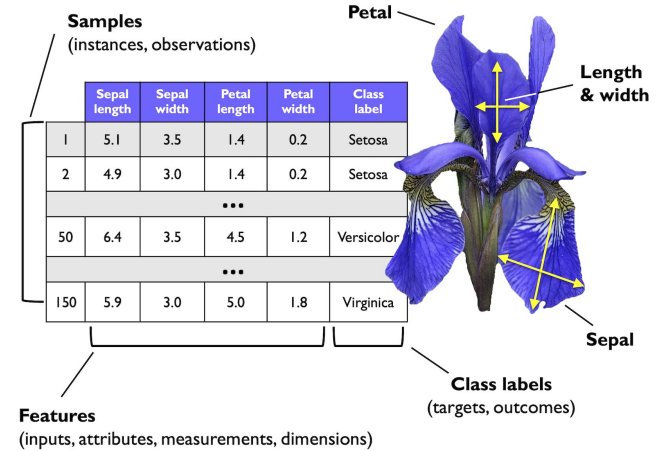
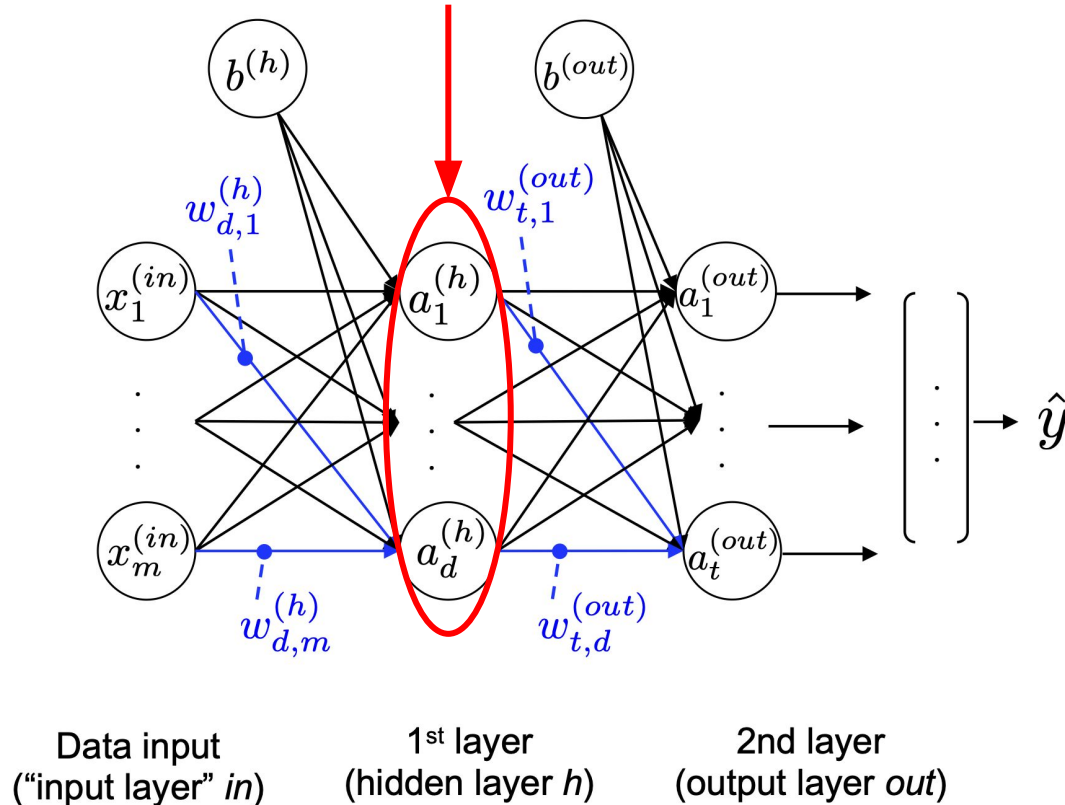
- Create linear combination
- Apply activation function

$$z_1^{(h)} = x_1^{(in)} w_{1,1}^{(h)} + x_2^{(in)} w_{1,2}^{(h)} + \dots + x_m^{(in)} w_{1,m}^{(h)}$$
$$a_1^{(h)} = \sigma(z_1^{(h)})$$

- Write in vector form

$$\mathbf{z}^{(h)} = \mathbf{x}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$
$$\mathbf{a}^{(h)} = \sigma(\mathbf{z}^{(h)})$$

Multi-Layer Neural Network



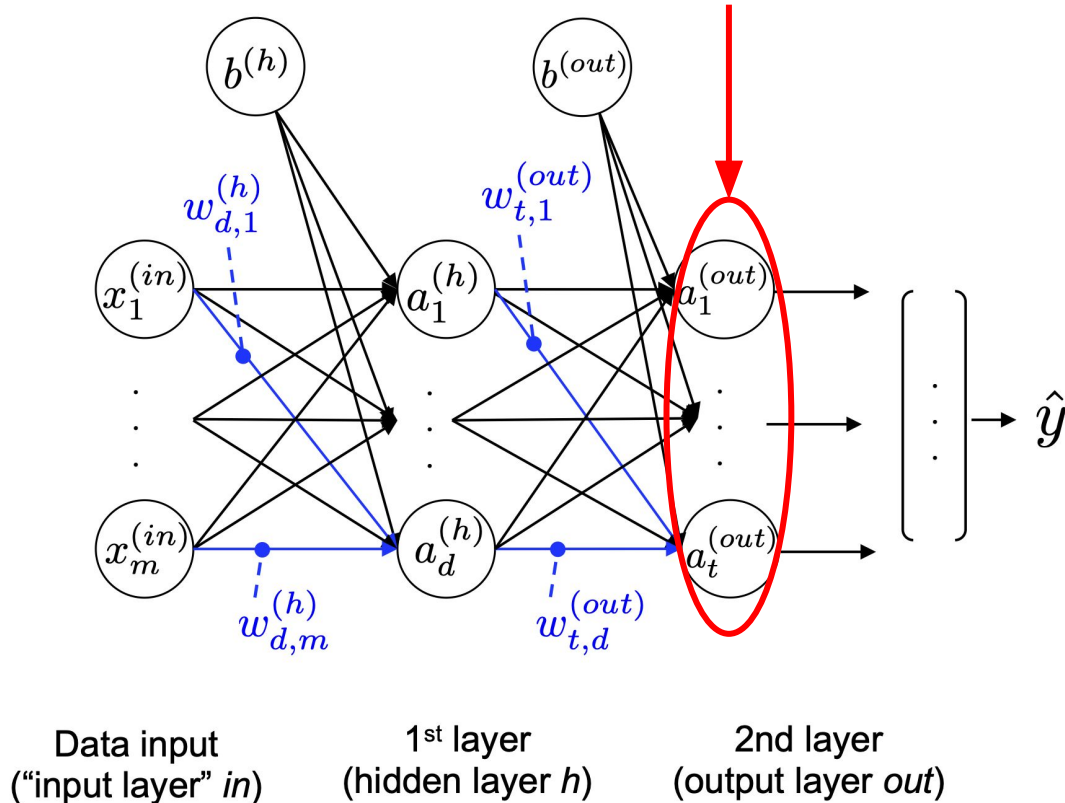
- Write in matrix form:

$$Z^{(h)} = X^{(in)} W^{(h)T} + b^{(h)}$$

$$A^{(h)} = \sigma(Z^{(h)})$$

This now includes all nodes of a layer

Multi-Layer Neural Network



- Write in matrix form:

$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)T} + \mathbf{b}^{(h)}$$

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)})$$

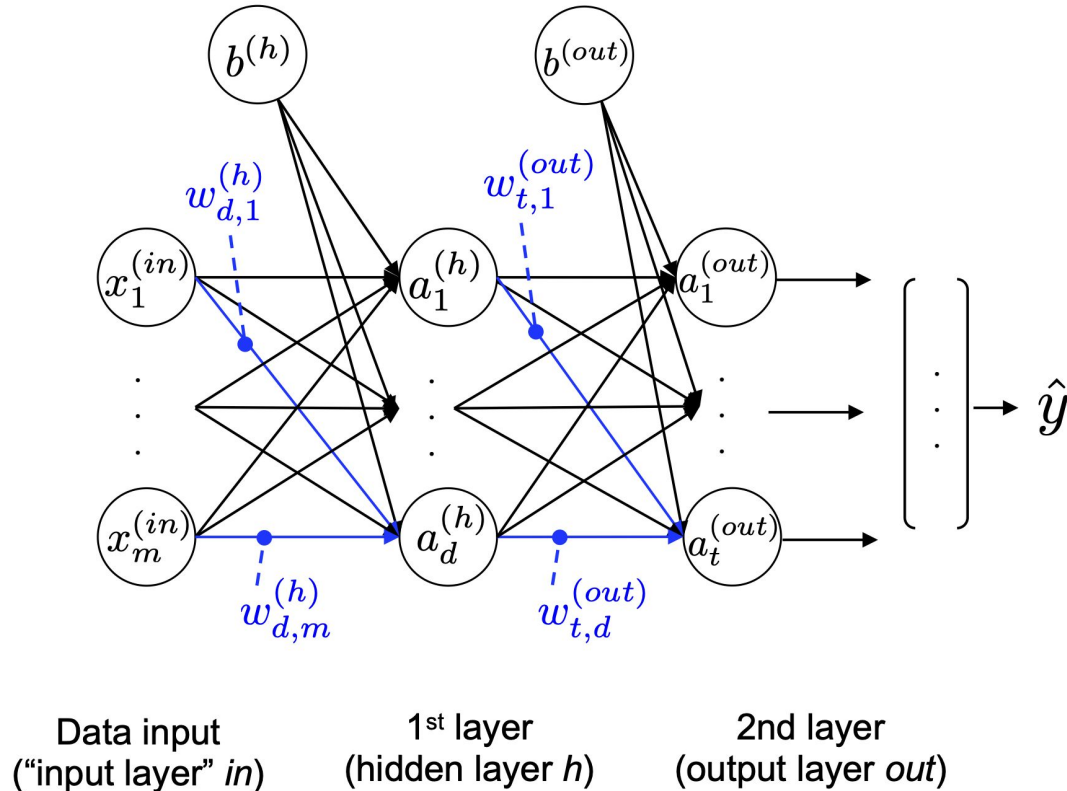
This now includes all nodes of a layer.

- Repeat the same for other hidden layers and the output layer

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)T} + \mathbf{b}^{(out)}$$

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)})$$

Multi-Layer Neural Network

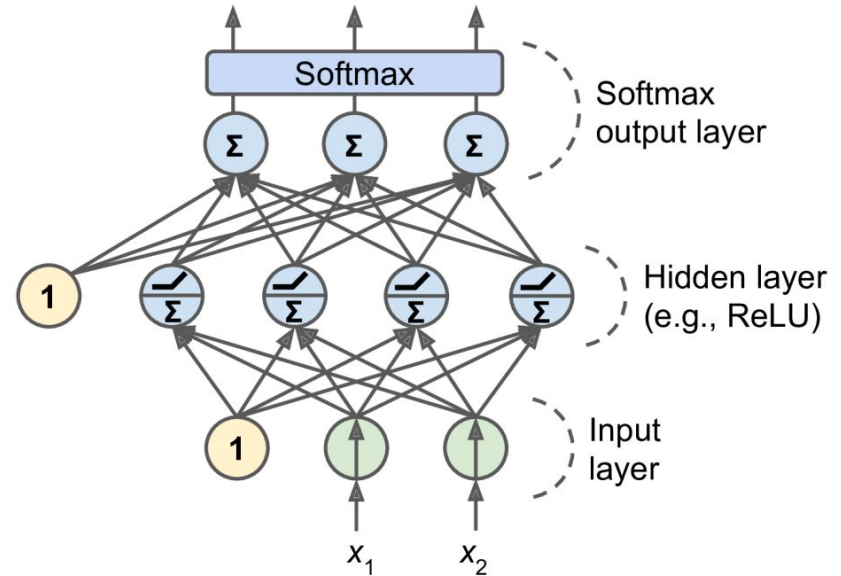
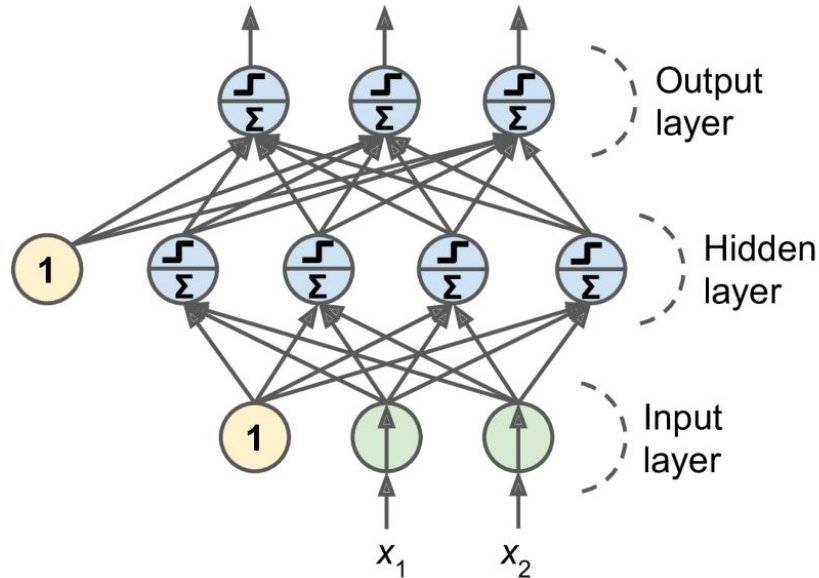


This neural network can now be evaluated with just two matrix operations in python

```
def forward(self, x):  
    # Hidden Layer  
  
    # input dim: [n_hidden, n_features]  
    #      dot [n_features, n_examples] .T  
    # output dim: [n_examples, n_hidden]  
    z_h = np.dot(x, self.weight_h.T) + self.bias_h  
    a_h = sigmoid(z_h)  
  
    # Output Layer  
    # input dim: [n_classes, n_hidden]  
    #      dot [n_hidden, n_examples] .T  
    # output dim: [n_examples, n_classes]  
    z_out = np.dot(a_h, self.weight_out.T) + self.bias_out  
    a_out = sigmoid(z_out)  
    return a_h, a_out
```

Multi-Layer Neural Network

We can also decide which kind of problem we want to solve e.g. regression, classification etc.



Jupyter Notebook Tutorial

Training a Neural Network

We now need to adjust the parameters (weights and biases) of the neural network to obtain our desired outcome i.e. represent a complex function.

This is done by minimizing an appropriate loss function:

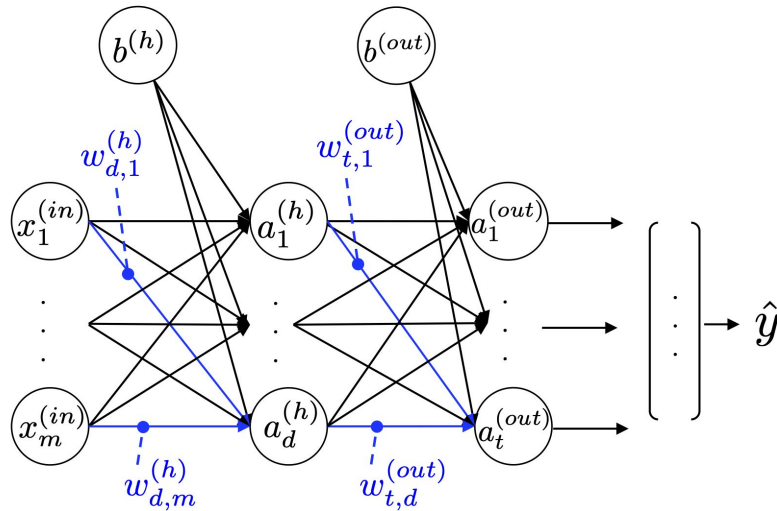
$$L(\mathbf{W}, \mathbf{b}) = \frac{1}{n} \sum_1^n \frac{1}{t} \sum_{j=1}^t (y_j^{[i]} - a_j^{(out)[i]})^2$$

where n and t are the size of the output layer and dataset respectively.

The output layer and the target variable looks something like this:

$$a^{(out)} = \begin{bmatrix} 0.1 \\ 0.9 \\ \vdots \\ 0.3 \end{bmatrix}, \quad y = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

Training a Neural Network: Backpropagation



Data input
("input layer" in)

1st layer
(hidden layer h)

2nd layer
(output layer out)

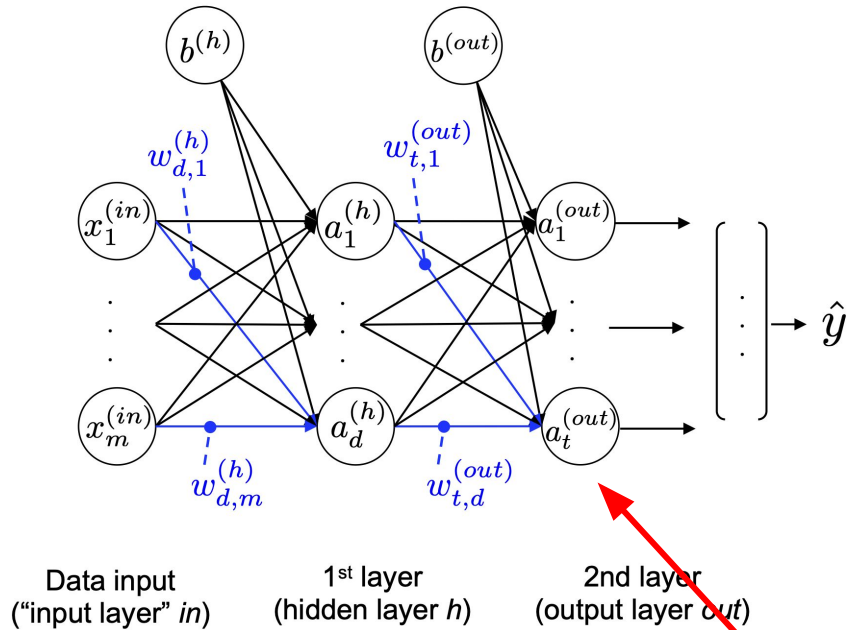
As we can see that output layer is a function of a function of a function of the input layer.

Therefore the loss function will also be so.

$$L(W, b) = f(g(h(u(v(x))))))$$

$$a_t^{out} = \sigma \left(b^{(out)} + \sum_{i=1}^d w_{t,i} a_i^{(h)} \right) = \sigma \left(b^{(out)} + \sum_{i=1}^d w_{t,i} \sigma \left(b^{(h)} + \sum_{j=1}^m w_{i,j} x_j \right) \right)$$

Training a Neural Network: Backpropagation



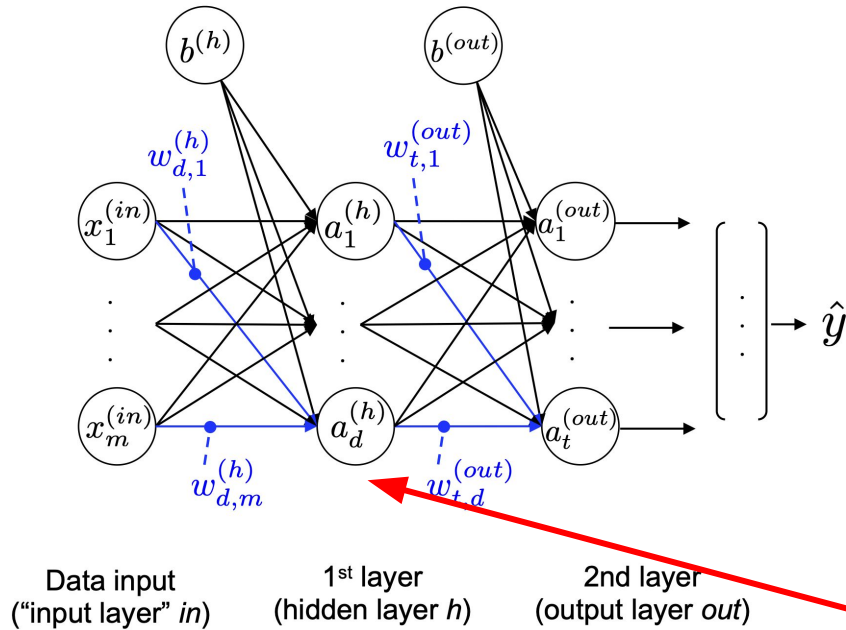
As we can see that output layer is a function of a function of a function of the input layer.

Therefore the loss function will also be so.

$$L(W, b) = f(g(h(u(v(x))))))$$

$$a_t^{out} = \sigma \left(b^{(out)} + \sum_{i=1}^d w_{t,i} a_i^{(h)} \right) = \sigma \left(b^{(out)} + \sum_{i=1}^d w_{t,i} \sigma \left(b^{(h)} + \sum_{j=1}^m w_{i,j} x_j \right) \right)$$

Training a Neural Network: Backpropagation



As we can see that output layer is a function of a function of a function of the input layer.

Therefore the loss function will also be so.

$$L(W, b) = f(g(h(u(v(x))))))$$

$$a_t^{out} = \sigma \left(b^{(out)} + \sum_{i=1}^d w_{t,i} a_i^{(h)} \right) = \sigma \left(b^{(out)} + \sum_{i=1}^d w_{t,i} \sigma \left(b^{(h)} + \sum_{j=1}^m w_{i,j} x_j \right) \right)$$

Training a Neural Network: Backpropagation

To train the neural network we need to minimize the loss function i.e. we need to set its gradient to zero.

As the loss function is a nested function we can compute its gradient through the chain rule:

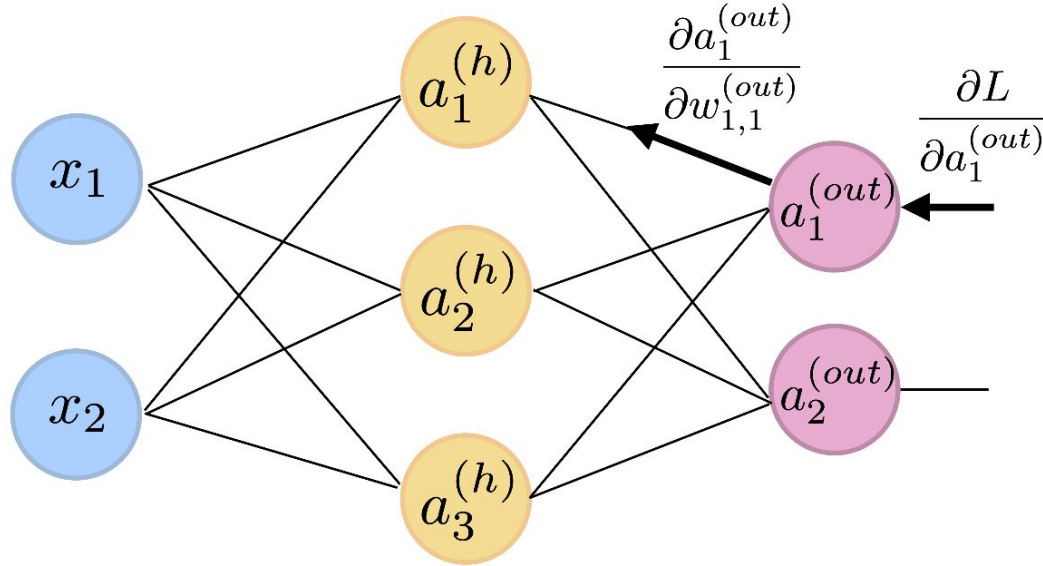
$$\frac{d}{dx} [f(g(x))] = \frac{df}{dg} \cdot \frac{dg}{dx}$$

Therefore the derivative of the loss function becomes

$$\frac{d}{dx} f(g(h(u(v(x))))) = \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}$$

Notice that we need to differentiate the outermost layer first and move backwards toward the input layer. This is where the name **backpropagation** comes from.

Training a Neural Network: Backpropagation



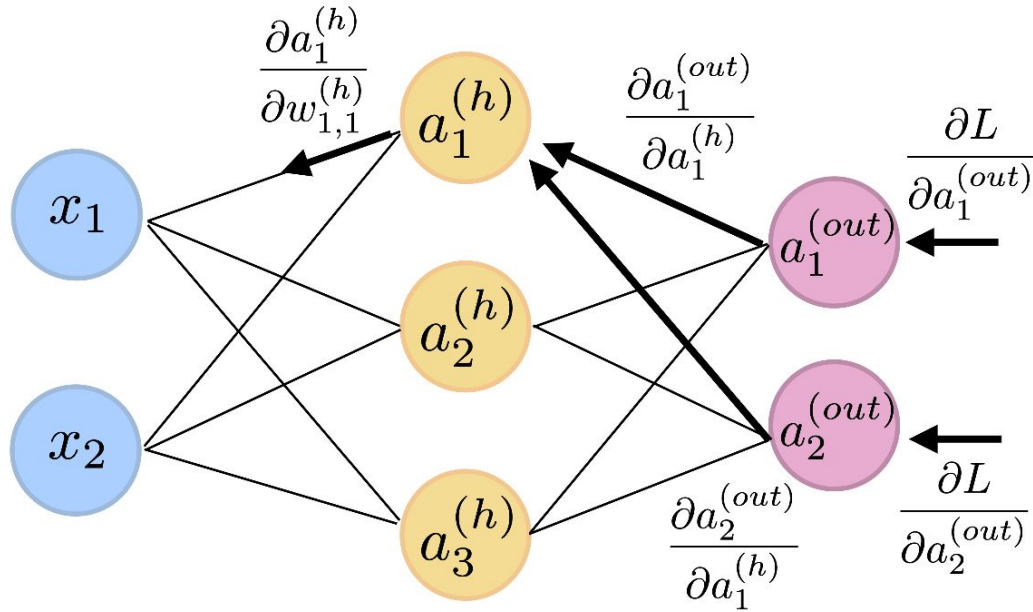
Gradient for output layer weight:

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

The value of $\frac{\partial a_1^{(out)}}{\partial z_1^{(out)}}$ is already known from the activation function. The $\frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$ is equal to the corresponding hidden layer value as z is a linear combination.

Training a Neural Network: Backpropagation



Gradient for hidden layer weight:

$$\frac{\partial L}{\partial w_{1,1}^{(h)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial w_{1,1}^{(h)}} + \frac{\partial L}{\partial a_2^{(out)}} \cdot \frac{\partial a_2^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial w_{1,1}^{(h)}}$$

Since the weight $w_{1,1}$ is connected to both output nodes, we have to use the multivariable chain rule to sum the two paths highlighted with bold arrows.

Automatic Differentiation

These calculations require lot of differentiation. Machine learning packages like PyTorch contains automatic differentiation implemented to handle them efficiently.

```
>>> w = torch.tensor(1.0, requires_grad=True)
>>> b = torch.tensor(0.5, requires_grad=True)
>>> x = torch.tensor([1.4])
>>> y = torch.tensor([2.1])
>>> z = torch.add(torch.mul(w, x), b)
>>> loss = (y-z).pow(2).sum()
>>> loss.backward()
>>> print('dL/dw : ', w.grad)
>>> print('dL/db : ', b.grad)
dL/dw :  tensor(-0.5600)
dL/db :  tensor(-0.4000)
>>> # verifying the computed gradient
>>> print(2 * x * ((w * x + b) - y))
tensor([-0.5600], grad_fn=<MulBackward0>)
```

Automatic Differentiation

These calculations require lot of differentiation. Machine learning packages like PyTorch contains automatic differentiation implemented to handle them efficiently.

```
>>> w = torch.tensor(1.0, requires_grad=True)
```

```
>>> b = torch.tensor(0.5, requires_grad=True)
```

```
>>> x = torch.tensor([1.4])
```

```
>>> y = torch.tensor([2.1])
```

```
>>> z = torch.add(torch.mul(w, x), b)
```

```
>>> loss = (y-z).pow(2).sum()
```

```
>>> loss.backward()
```

```
>>> print('dL/dw : ', w.grad)
```

```
>>> print('dL/db : ', b.grad)
```

```
dL/dw : tensor(-0.5600)
```

```
dL/db : tensor(-0.4000)
```

```
>>> # verifying the computed gradient
```

```
>>> print(2 * x * ((w * x + b) - y))
```

```
tensor([-0.5600], grad_fn=<MulBackward0>)
```



Ask for automatic
differentiation to be
performed

Automatic Differentiation

These calculations require lot of differentiation. Machine learning packages like PyTorch contains automatic differentiation implemented to handle them efficiently.

```
>>> w = torch.tensor(1.0, requires_grad=True)
>>> b = torch.tensor(0.5, requires_grad=True)
>>> x = torch.tensor([1.4])
>>> y = torch.tensor([2.1])
>>> z = torch.add(torch.mul(w, x), b)
>>> loss = (y-z).pow(2).sum()
>>> loss.backward()
>>> print('dL/dw : ', w.grad)
>>> print('dL/db : ', b.grad)
dL/dw :  tensor(-0.5600)
dL/db :  tensor(-0.4000)
>>> # verifying the computed gradient
>>> print(2 * x * ((w * x + b) - y))
tensor([-0.5600], grad_fn=<MulBackward0>)
```

Evaluate the function

Automatic Differentiation

These calculations require lot of differentiation. Machine learning packages like PyTorch contains automatic differentiation implemented to handle them efficiently.

```
>>> w = torch.tensor(1.0, requires_grad=True)
>>> b = torch.tensor(0.5, requires_grad=True)
>>> x = torch.tensor([1.4])
>>> y = torch.tensor([2.1])
>>> z = torch.add(torch.mul(w, x), b)
>>> loss = (y-z).pow(2).sum()
>>> loss.backward() ← Evaluate the gradient
>>> print('dL/dw : ', w.grad)
>>> print('dL/db : ', b.grad)
dL/dw :  tensor(-0.5600)
dL/db :  tensor(-0.4000)
>>> # verifying the computed gradient
>>> print(2 * x * ((w * x + b) - y))
tensor([-0.5600], grad_fn=<MulBackward0>)
```

Automatic Differentiation

These calculations require lot of differentiation. Machine learning packages like PyTorch contains automatic differentiation implemented to handle them efficiently.

```
>>> w = torch.tensor(1.0, requires_grad=True)
>>> b = torch.tensor(0.5, requires_grad=True)
>>> x = torch.tensor([1.4])
>>> y = torch.tensor([2.1])
>>> z = torch.add(torch.mul(w, x), b)
>>> loss = (y-z).pow(2).sum()
>>> loss.backward()
>>> print('dL/dw : ', w.grad)
>>> print('dL/db : ', b.grad)
dL/dw :  tensor(-0.5600)
dL/db :  tensor(-0.4000)
>>> # verifying the computed gradient
>>> print(2 * x * ((w * x + b) - y))
tensor([-0.5600], grad_fn=<MulBackward0>)
```

Automatically computed gradient matches with the real analytical gradient

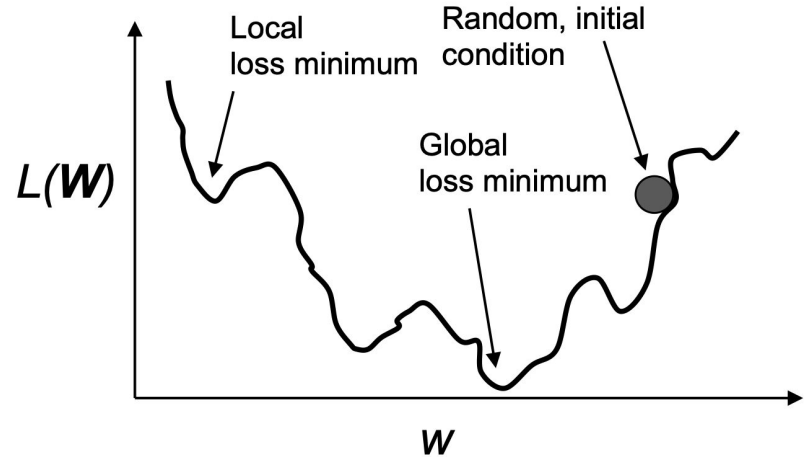
Minimizing Loss Function: Optimizers

The next step is to update the weights and biases to minimize the loss function. For example gradient descent:

$$w_{1,1}^{(out)} := w_{1,1}^{(out)} - \eta \frac{\partial L}{\partial w_{1,1}^{(out)}}$$

However, the Loss function in neural networks contains many local minima and plateaus.

We need better optimization algorithms.



Minimizing Loss Function: Optimizers

Momentum Optimization

Imagine a bowling ball rolling down a gentle slope on a smooth surface: it will start out slowly, but it will quickly pick up momentum.

In contrast, regular Gradient Descent will simply take small regular steps down the slope, so it will take much more time to reach the bottom.

$$1. \quad \mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta)$$

$$2. \quad \theta \leftarrow \theta + \mathbf{m}$$

Improvements on Momentum Optimization

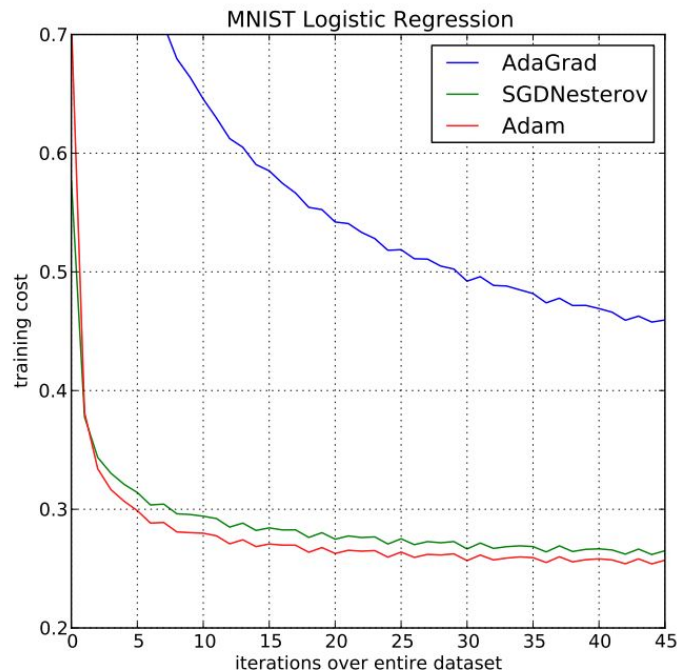
- Nesterov Accelerated Gradient
- AdaGrad
- RMSProp
- Adaptive moment estimation (Adam)
- Nesterov adaptive moment estimation (Nadam)

Minimizing Loss Function: Optimizers

Adam Optimizer

1. $\mathbf{m} \leftarrow \beta_1 \mathbf{m} - (1 - \beta_1) \nabla_{\theta} J(\theta)$
2. $\mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
3. $\widehat{\mathbf{m}} \leftarrow \frac{\mathbf{m}}{1 - \beta_1^t}$
4. $\widehat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \beta_2^t}$
5. $\theta \leftarrow \theta + \eta \widehat{\mathbf{m}} \oslash \sqrt{\widehat{\mathbf{s}} + \epsilon}$

Most commonly used due to superior performance



Adam: A Method for Stochastic Optimization
by Diederik P. Kingma and Jimmy Lei Ba,
2014 (<https://arxiv.org/abs/1412.6980>)

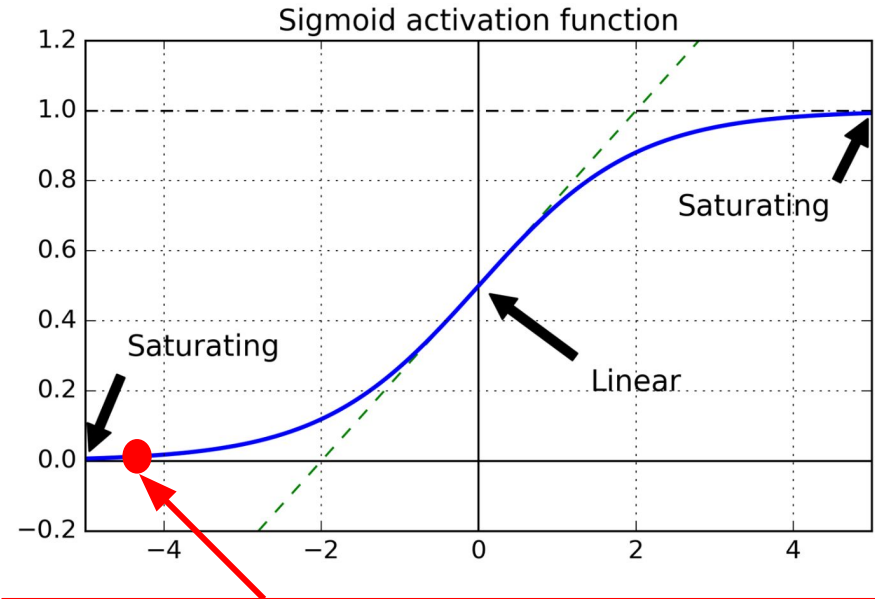
Vanishing Gradient Problem

Gradients often get smaller and smaller as the backpropagation training progresses leaving weights virtually unchanged, and never converging to a good solution.

$$\frac{d}{dx} [f(g(x))] = \frac{df}{dg} \cdot \frac{dg}{dx} = 0$$

Reason: Saturating Activation functions have near zero gradient when the argument is large.

Solution: Use non saturating activation functions e.g. Leaky ReLU

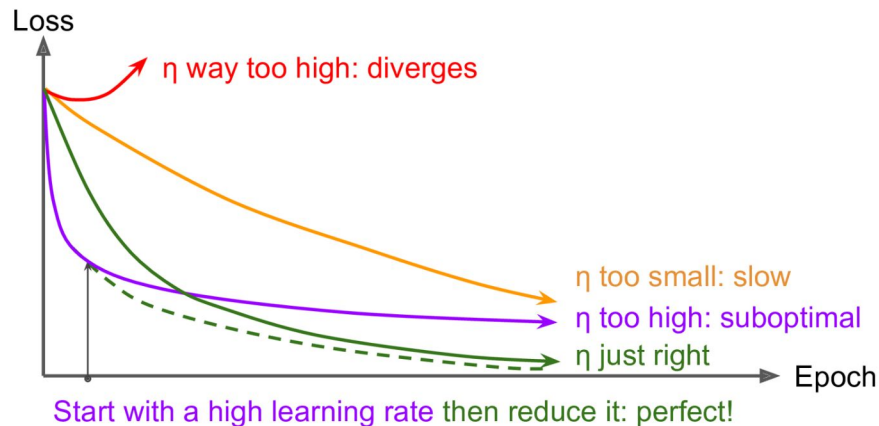


Extreme issue when using logistic function with weights sampled from a Gaussian distribution centered at zero.

Learning Rate Scheduling

Finding a good learning rate can be tricky. If you set it way too high, training may actually diverge. If you set it too low, training will take a very long time.

If you **start with a high learning rate and then reduce it once it stops making fast progress**, you can reach a good solution faster than using a constant learning rate.



Power Scheduling:

$$\eta(t) = \eta_0 / (1 + t/k)^c$$

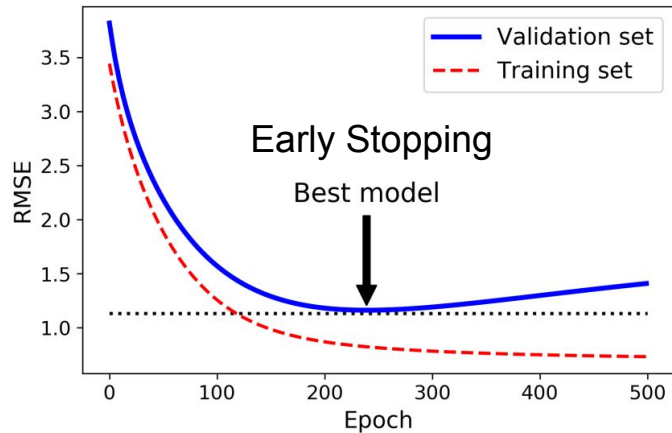
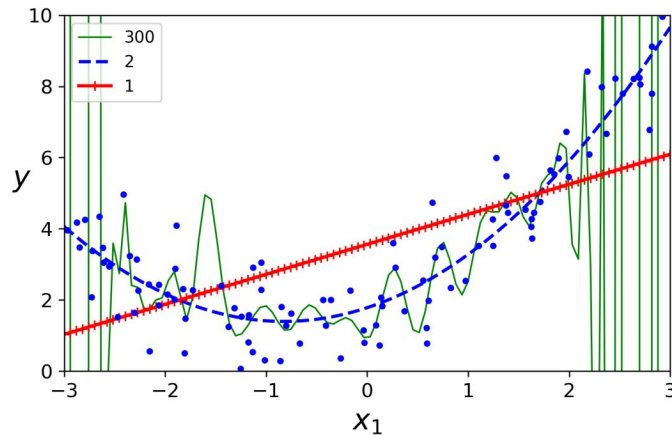
Exponential Scheduling:

$$\eta(t) = \eta_0 0.1^{t/s}$$

Regularization

As neural networks contain many ($>10^3$) parameters, they are highly prone to overfitting problem. Always use some form of regularization.

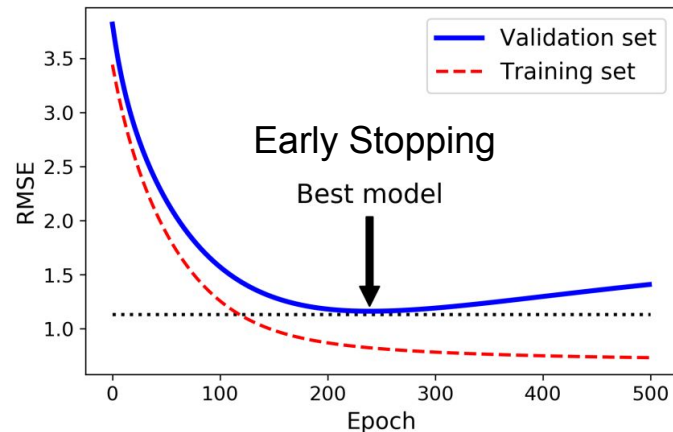
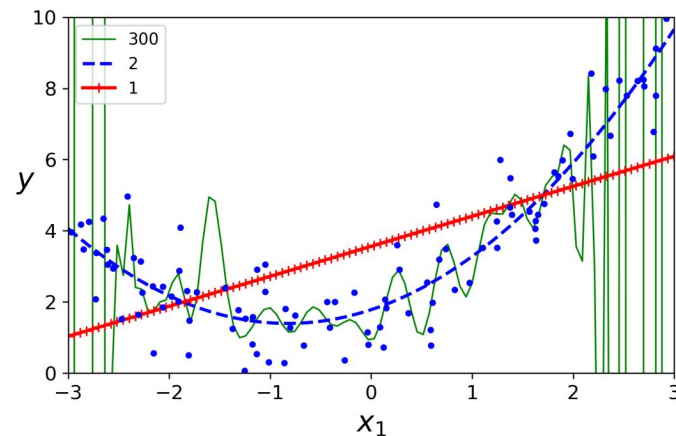
1. **Weight decay** (L2 penalty): $Loss + \alpha \sum ||w_i||^2$
2. **Lasso** (L1 penalty): $Loss + \alpha \sum ||w_i||$ (exclude unnecessary parameters)
3. **Early Stopping**: Stop training when Validation Loss stops decreasing
4. **Dropout**: Exclude random nodes during training



Regularization

As neural networks contain many ($>10^3$) parameters, they are highly prone to overfitting problem. Always use some form of regularization.

1. **Weight decay** (L2 penalty): $\text{Loss} + \alpha \sum ||w_i||^2$
2. **Lasso** (L1 penalty): $\text{Loss} + \alpha \sum ||w_i||$ (exclude unnecessary parameters)
3. **Early Stopping**: Stop training when Validation Loss stops decreasing
4. **Dropout**: Exclude random nodes during training

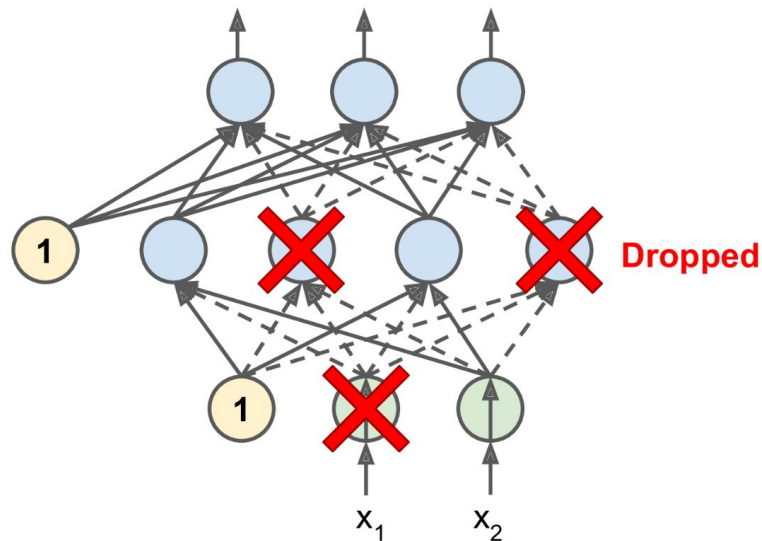


Dropout Regularization

At every training step, every neuron (excluding the output neurons) has a probability p of being temporarily “dropped out,” meaning it will be **entirely ignored during this training step**, but it may be active during the next step. After training, neurons don’t get dropped anymore.

Neurons trained with dropout

- cannot co-adapt with neighbours and have to be as useful as possible on their own.
- cannot rely excessively on just a few input neurons

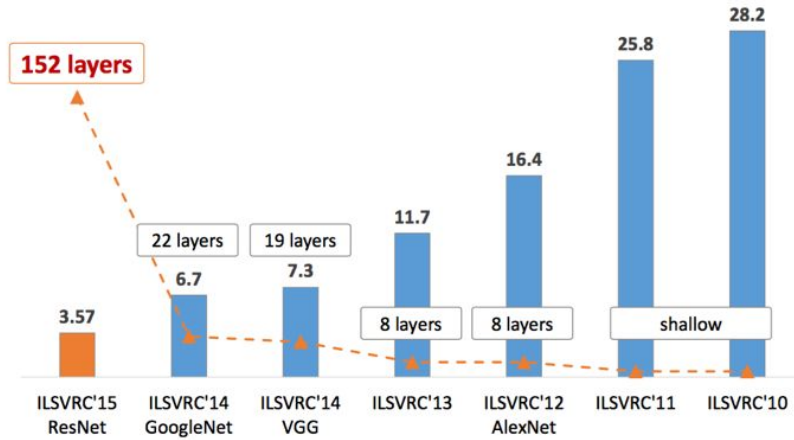
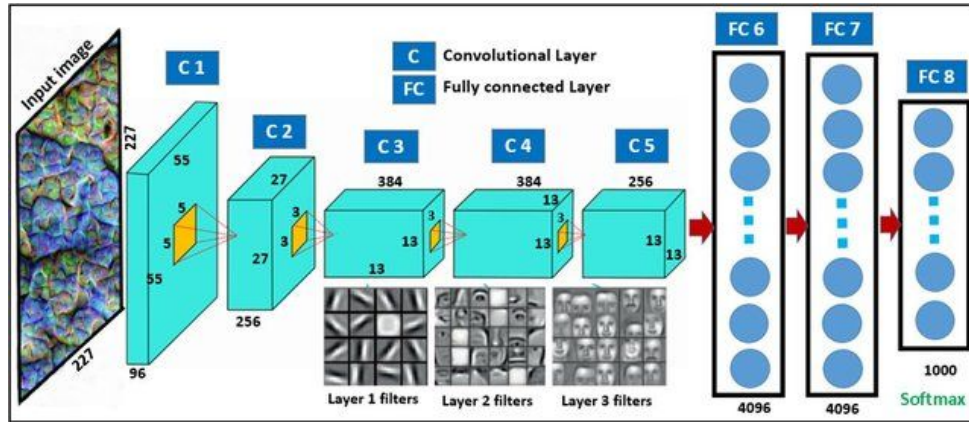


Jupyter Notebook Tutorial

Deep Learning from Neural Network

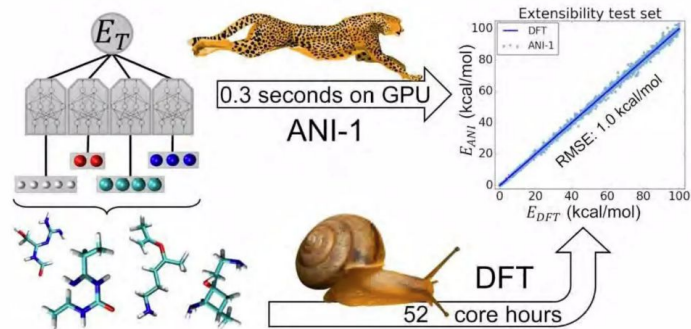
Deep Learning: Using multiple hidden layers in a neural network improves its performance significantly.

But it makes training memory intensive and time consuming. GPUs revolutionized this field by parallelizing the optimization process over many cores.



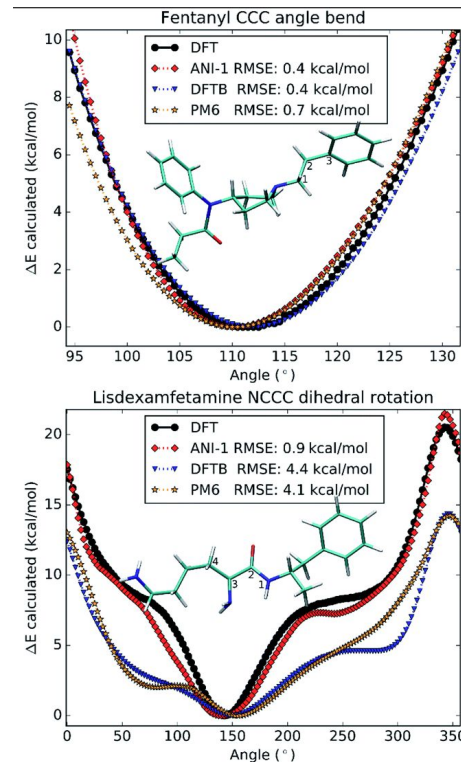
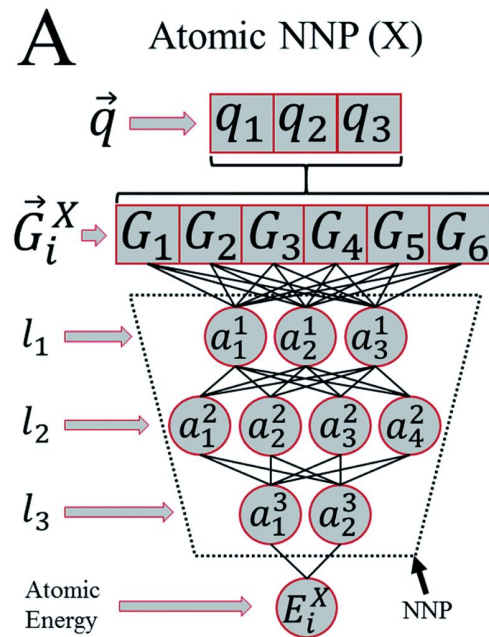
AlexNet: Krizhevsky et. al., 2012, Hassan and Goussev 2019, Das 2017

Application of Deep Learning in Chemistry

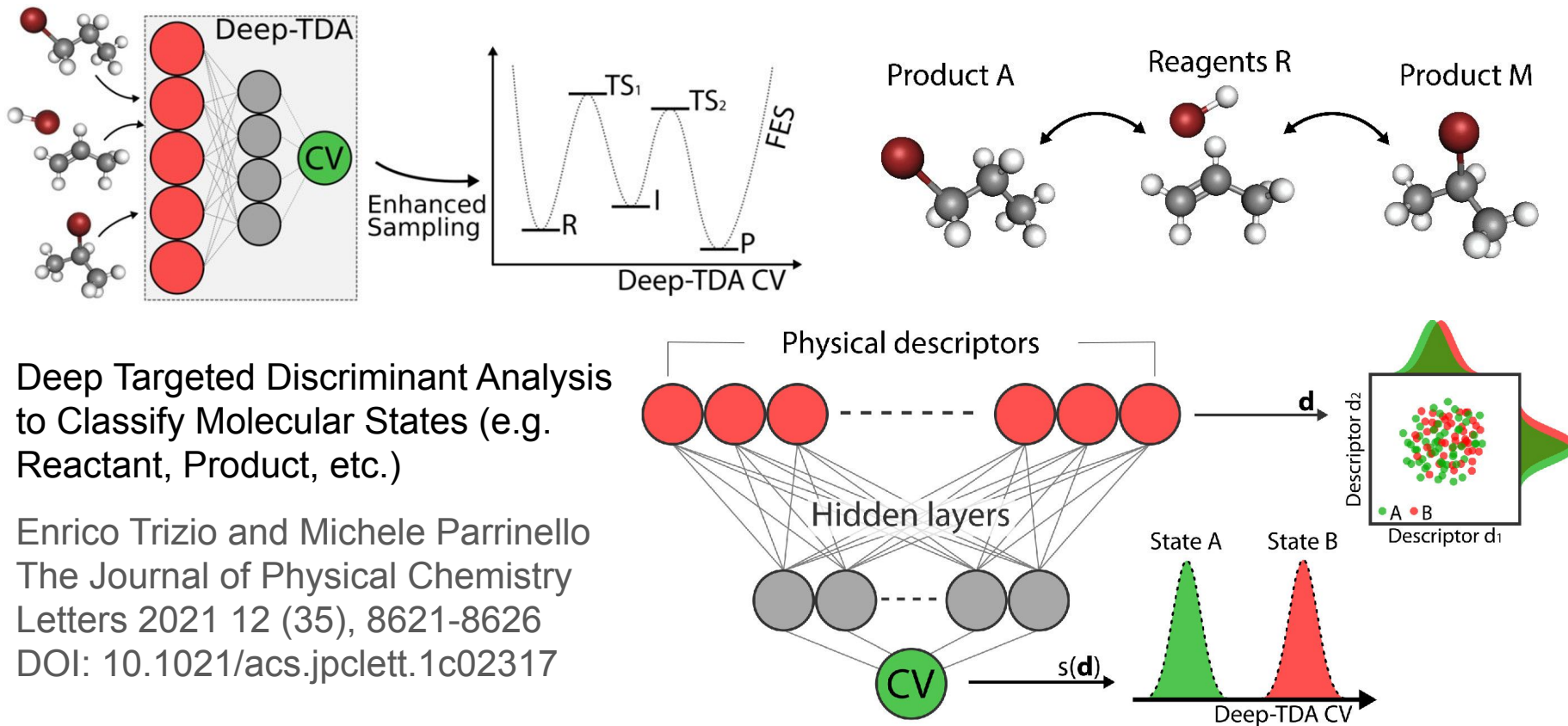


Smith, J. S., Isayev, O., & Roitberg, A. E. (2017). ANI-1: an extensible neural network potential with DFT accuracy at force field computational cost. *Chemical science*, 8(4), 3192-3203.

<https://doi.org/10.1039/C6SC05720A>



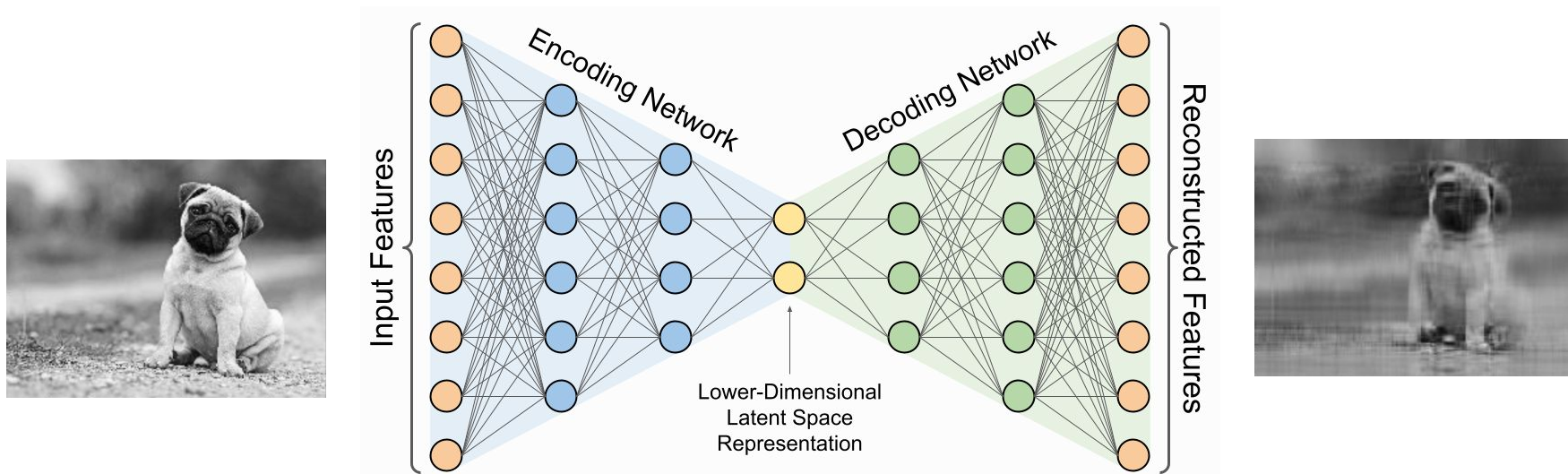
Application of Deep Learning in Chemistry



Deep Targeted Discriminant Analysis to Classify Molecular States (e.g. Reactant, Product, etc.)

Enrico Trizio and Michele Parrinello
The Journal of Physical Chemistry Letters 2021 12 (35), 8621-8626
DOI: 10.1021/acs.jpcllett.1c02317

Dimensionality Reduction: Autoencoder

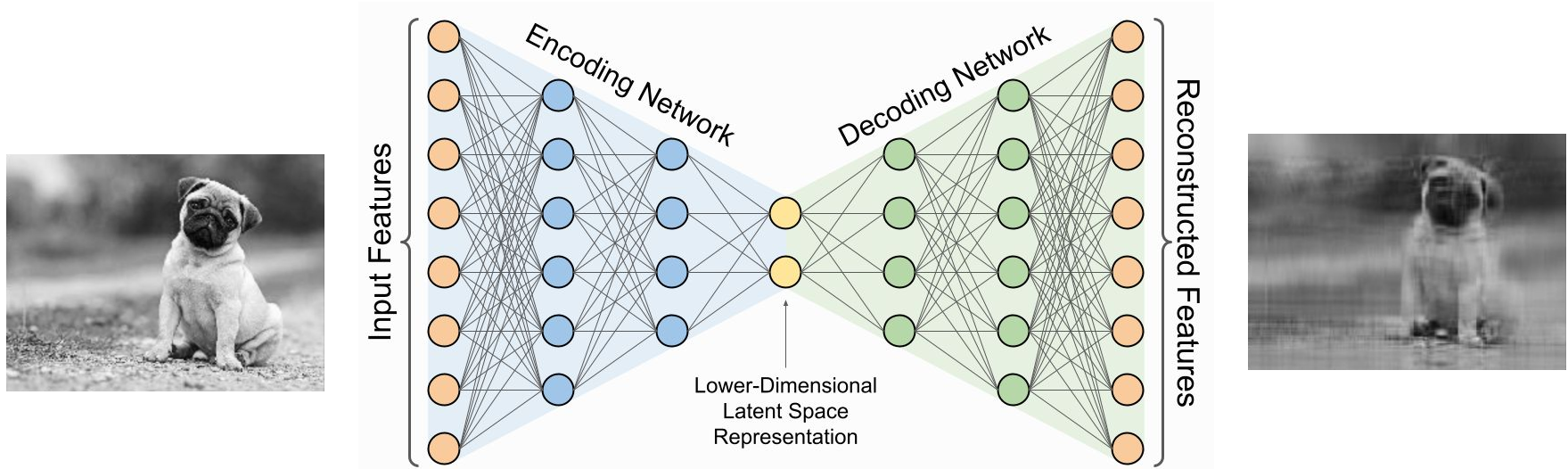


First an **encoder** compresses the input data into a lower dimensional latent space.

Then a **decoder** attempts to reconstruct the input data from the latent space.

To perform a better reconstruction the **latent space should contain as much information as possible about the input data.**

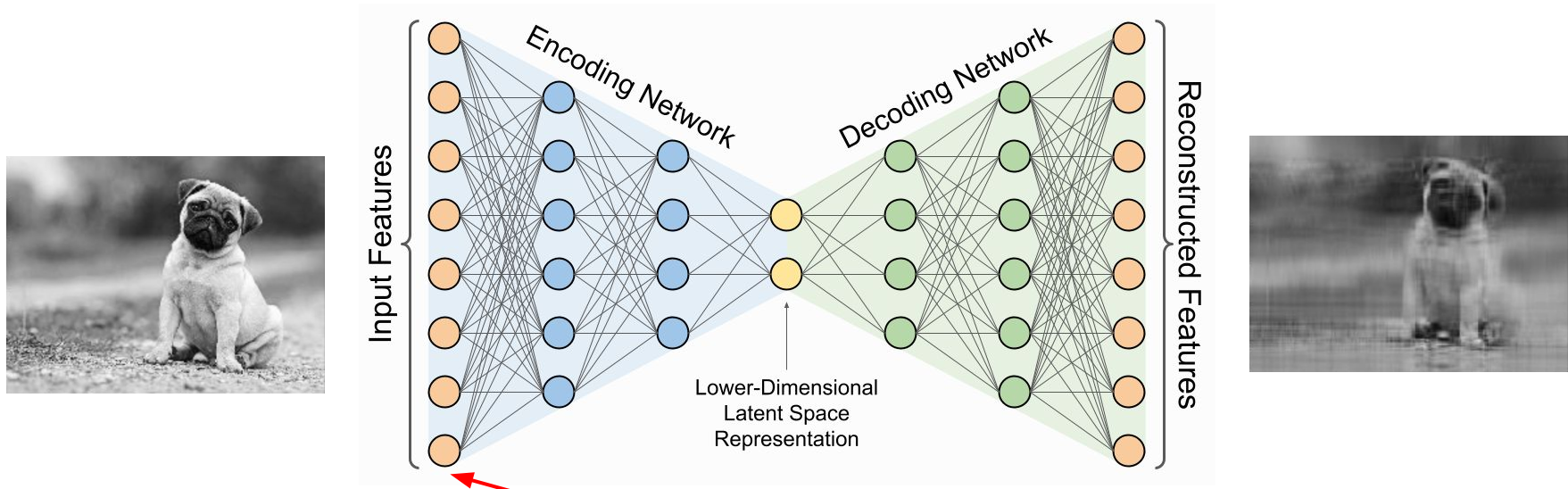
Dimensionality Reduction: Autoencoder



To perform a better reconstruction the **latent space should contain as much information as possible about the input data**. This is achieved by minimizing the reconstruction loss:

$$\mathcal{L}_{\text{reconstruction}} = \sum_{\text{samples}} \sum_{\text{features}} \left(f_{\text{decoder}}(f_{\text{encoder}}(x_i)) - x_i \right)^2$$

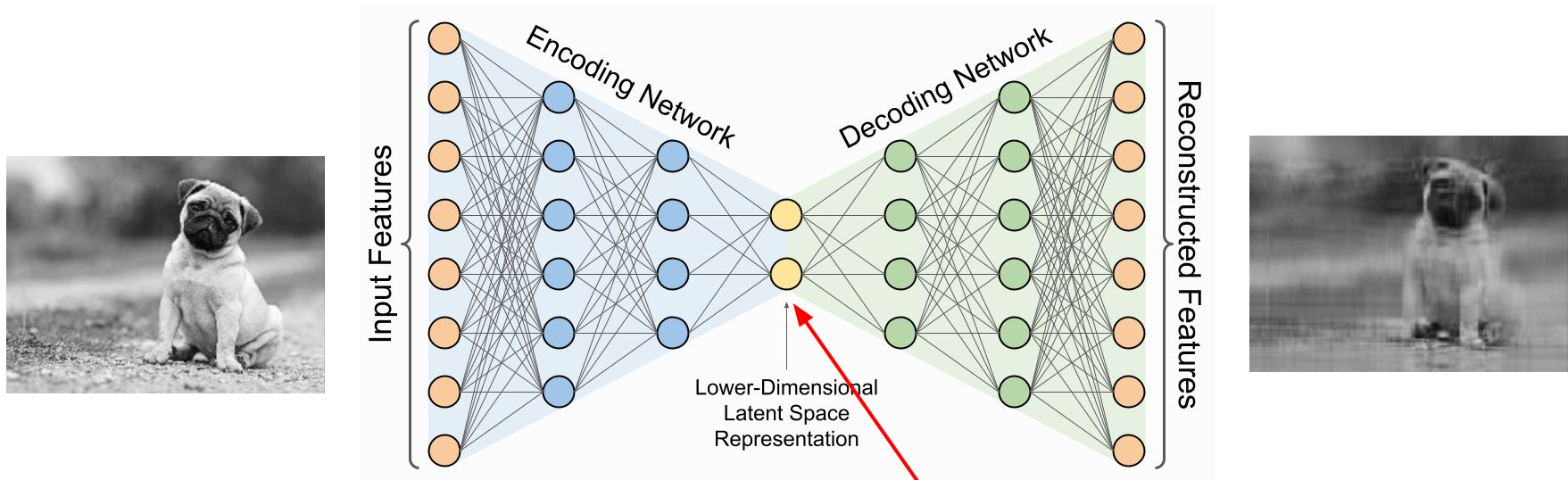
Dimensionality Reduction: Autoencoder



To perform a better reconstruction the latent space should contain as much information as possible about the input data. This is achieved by minimizing the reconstruction loss:

$$\mathcal{L}_{\text{reconstruction}} = \sum_{\text{samples}} \sum_{\text{features}} \left(f_{\text{decoder}}(f_{\text{encoder}}(x_i)) - x_i \right)^2$$

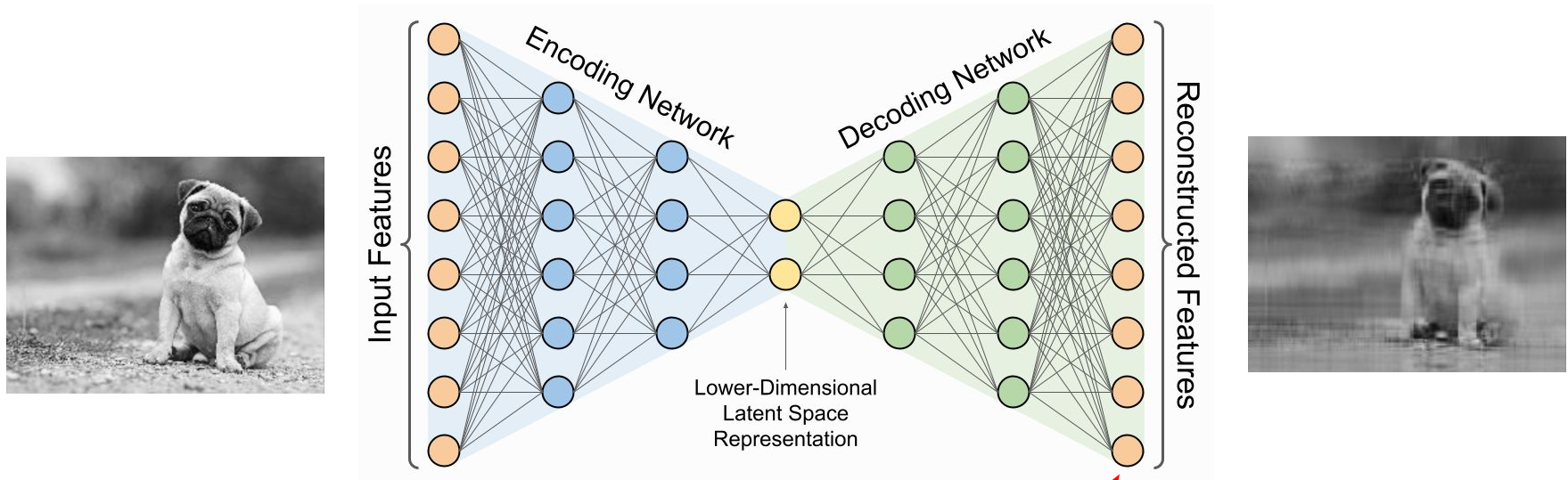
Dimensionality Reduction: Autoencoder



To perform a better reconstruction the latent space should contain as much information as possible about the input data. This is achieved by minimizing the reconstruction loss:

$$\mathcal{L}_{\text{reconstruction}} = \sum_{\text{samples}} \sum_{\text{features}} \left(f_{\text{decoder}}(f_{\text{encoder}}(x_i)) - x_i \right)^2$$

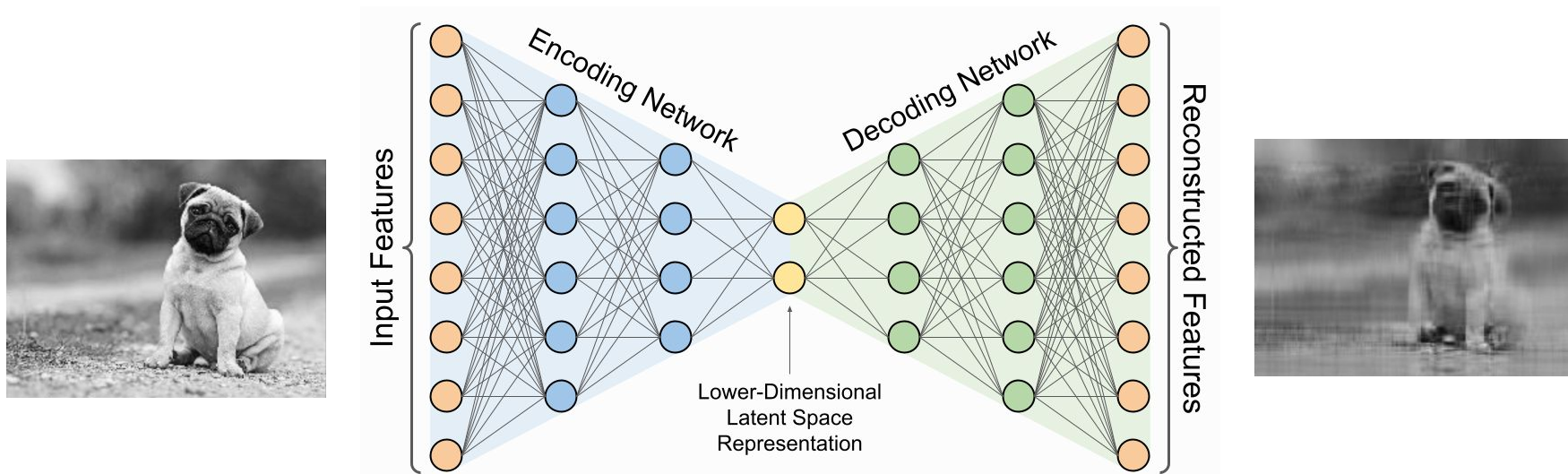
Dimensionality Reduction: Autoencoder



To perform a better reconstruction the latent space should contain as much information as possible about the input data. This is achieved by minimizing the reconstruction loss:

$$\mathcal{L}_{\text{reconstruction}} = \sum_{\text{samples}} \sum_{\text{features}} \left(f_{\text{decoder}}(f_{\text{encoder}}(x_i)) - x_i \right)^2$$

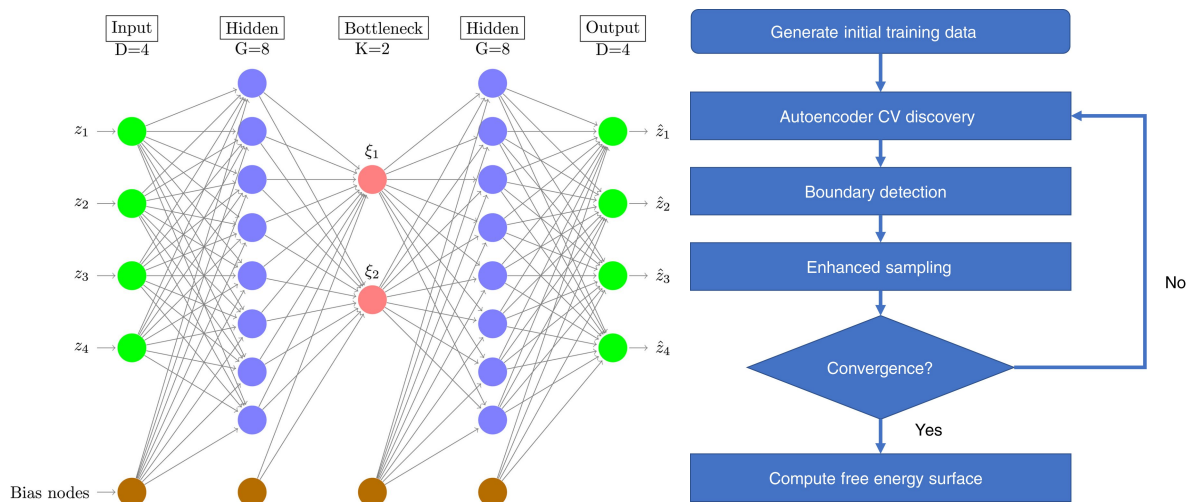
Dimensionality Reduction: Autoencoder



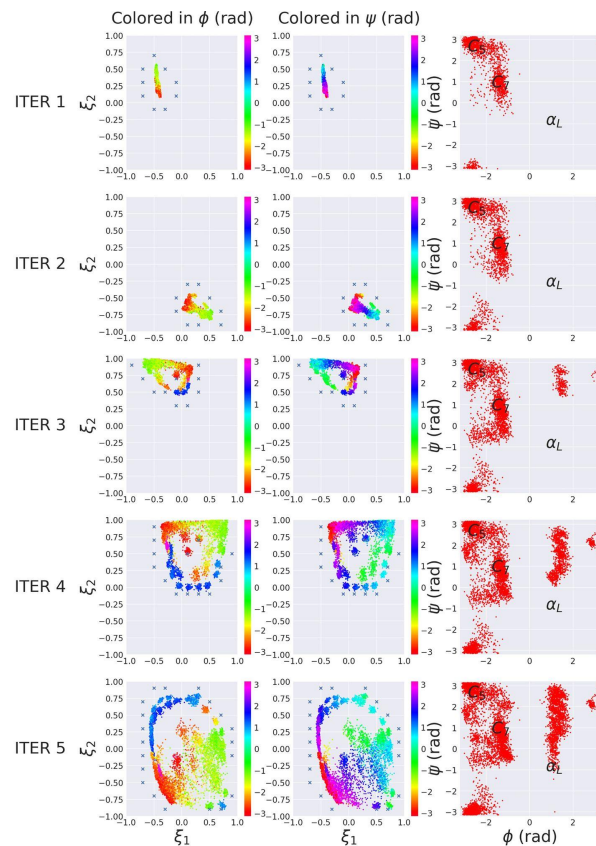
The **latent space** is used as the reduced dimensional representation of the data.

$$\mathcal{L}_{\text{reconstruction}} = \sum_{\text{samples}} \sum_{\text{features}} \left(f_{\text{decoder}}(f_{\text{encoder}}(x_i)) - x_i \right)^2$$

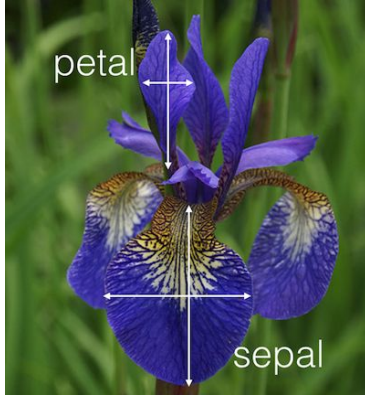
Application of Autoencoder in Chemistry



Chen, W., & Ferguson, A. L. (2018). Molecular enhanced sampling with autoencoders: On-the-fly collective variable discovery and accelerated free energy landscape exploration. *Journal of computational chemistry*, 39(25), 2079-2102. <https://doi.org/10.1002/jcc.25520>

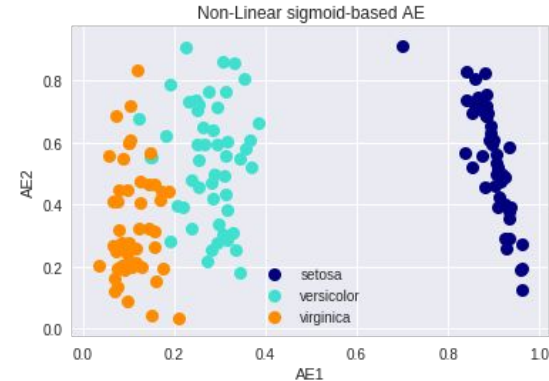
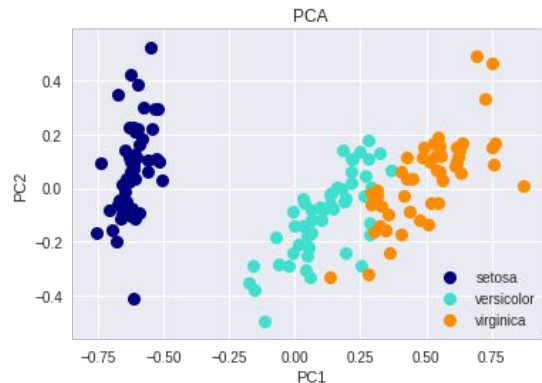


Dimensionality Reduction: Autoencoder



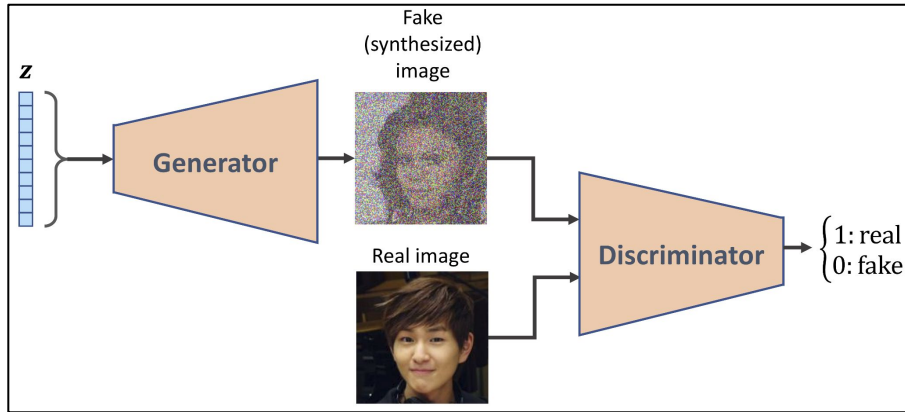
Example of Autoencoder applied to Iris dataset

	sepal length in cm	sepal width in cm	petal length in cm	petal width in cm	class label
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica



Combinations of Neural Networks

Most practical application of deep learning now involves a combination of several neural networks connected to each other. The “architecture” is optimized to perform specific tasks.



Generative Adversarial Network (GAN)

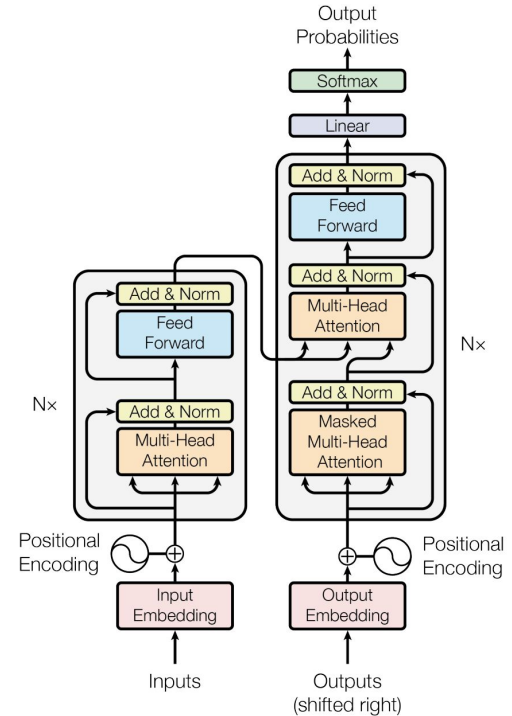


Figure 1: The Transformer - model architecture.

Transformer (used in e.g. ChatGPT)